

Lecture 3: Deep Neural Networks

Intro to Neural Networks

Firuz Kamalov

MAI 604: Deep Learning

January 30, 2026

Lecture Goals

The primary goals of today's lecture are to understand:

- 1 The architecture of deep networks.
- 2 The matrix notation of deep networks.
- 3 The advantages of deep networks over shallow networks (efficiency and linear regions).

Contents

- ① Composing neural networks
- ② From composition to deep networks
- ③ Matrix Notation & General Formulation
- ④ Shallow vs. Deep Networks

Contents

- 1 Composing neural networks
- 2 From composition to deep networks
- 3 Matrix Notation & General Formulation
- 4 Shallow vs. Deep Networks

- **Shallow Neural Networks:** Have a single hidden layer.
- **Deep Neural Networks:** Have more than one hidden layer.
- With ReLU activations, both describe **piecewise linear mappings**.
- **Key Insight:** Deep networks can produce many more linear regions than shallow networks for a given number of parameters.

Composing Two Shallow Networks

Consider passing the output of one shallow network into another.

Network 1: Inputs x , outputs y .

$$h_1 = a[\theta_{10} + \theta_{11}x]$$

$$h_2 = a[\theta_{20} + \theta_{21}x]$$

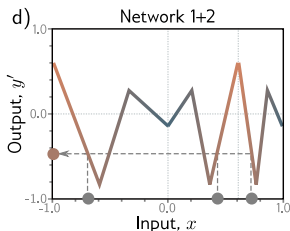
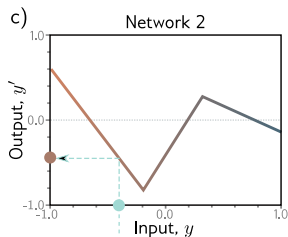
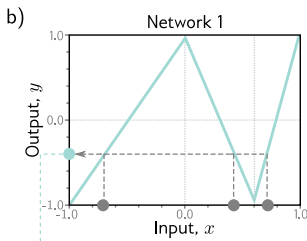
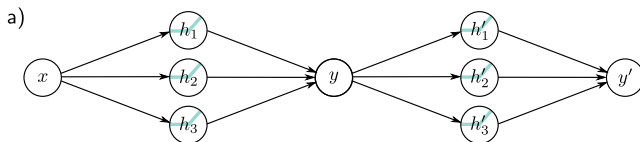
$$h_3 = a[\theta_{30} + \theta_{31}x]$$

$$y = \phi_0 + \phi_1 h_1 + \phi_2 h_2 + \phi_3 h_3$$

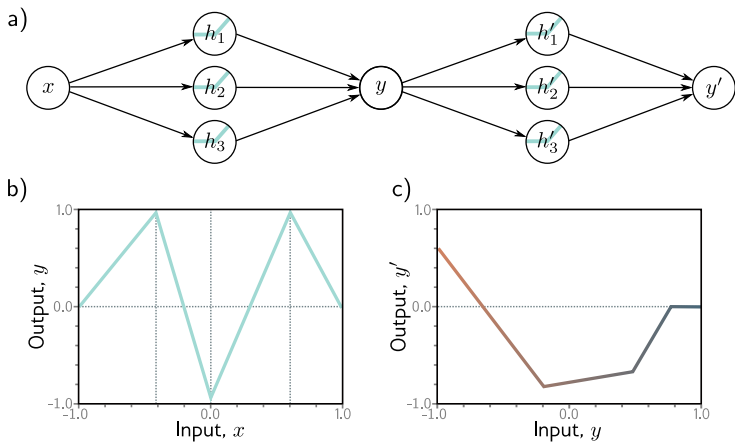
Network 2: Inputs y , outputs y' .

$$h'_1 = a[\theta'_{10} + \theta'_{11}y] \dots$$

Visualizing Composition



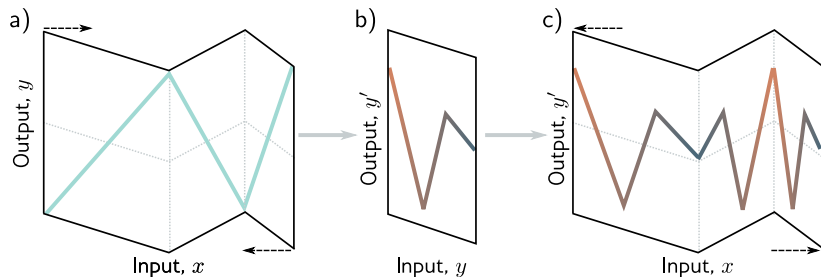
Concept Check



Consider the composition of these networks. What happens to the input space x as it passes through the first network to become y ?

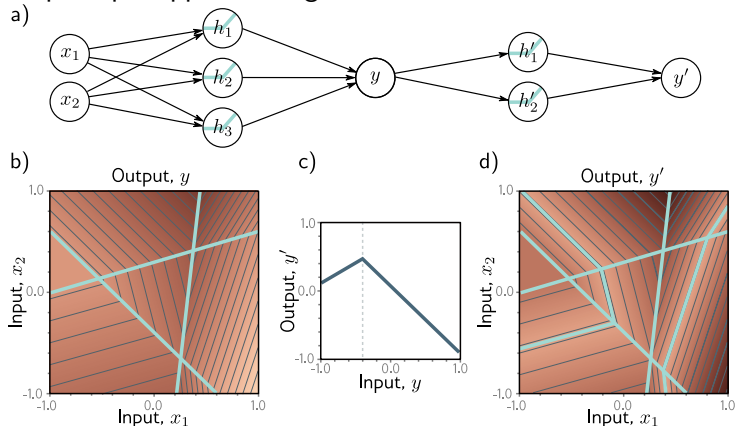
Folding the Input Space

Intuition: The first network "**folds**" the input space back onto itself.



Composing with 2D Inputs

The same principle applies to higher dimensions.



- (b) The first network produces a function consisting of 7 linear regions
- (d) Each of the regions in (b) is divided into two new regions by the second network to create a total of 13 linear regions

Contents

- ① Composing neural networks
- ② From composition to deep networks
- ③ Matrix Notation & General Formulation
- ④ Shallow vs. Deep Networks

Composition is a Deep Network

The composition of two shallow networks is a special case of a deep network with two hidden layers.

Substitute the output y of Net 1 into the equations for Net 2:

$$h'_1 = a[\theta'_{10} + \theta'_{11}(\phi_0 + \phi_1 h_1 + \phi_2 h_2 + \phi_3 h_3)]$$

This can be rewritten as a standard layer transition:

$$h'_1 = a[\psi_{10} + \psi_{11} h_1 + \psi_{12} h_2 + \psi_{13} h_3]$$

where ψ parameters absorb the weights θ and ϕ .

The Two-Layer Network

A general deep network with two hidden layers allows arbitrary weights ψ , not just those formed by the product of two shallow nets.

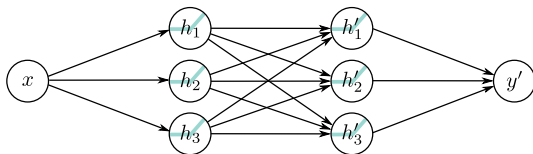
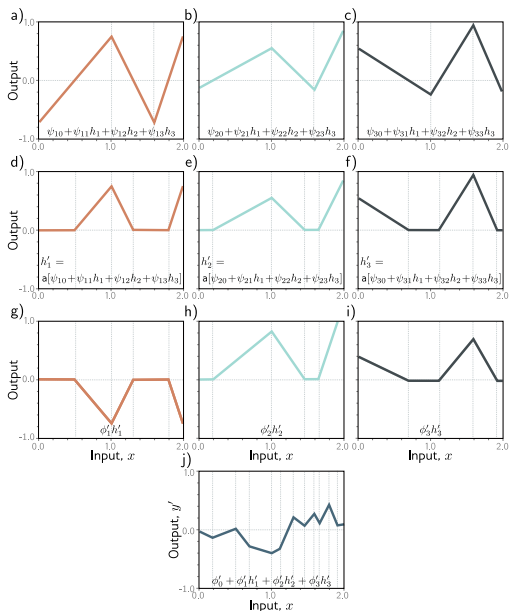


Figure 1: NN parameters: 1 input, 2 hidden layers, 3 hidden units, 1 output

$$y' = \phi'_0 + \phi'_1 h'_1 + \phi'_2 h'_2 + \phi'_3 h'_3$$

Another View: Clipping and Recombining



Neural Network Architecture (2 Inputs)

Calculate the output y for input $x_1 = -1, x_2 = 0.5$.

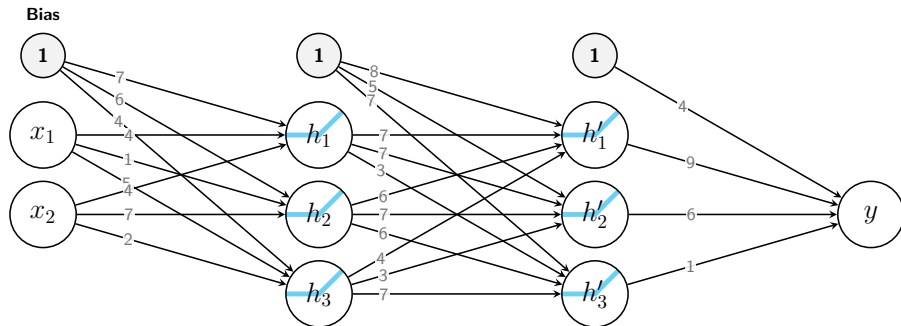


Figure 2: Deep network with 2 inputs

Neural Network Architecture (2 Inputs)

Calculate the output y for input $x_1 = -1, x_2 = 0.5$.

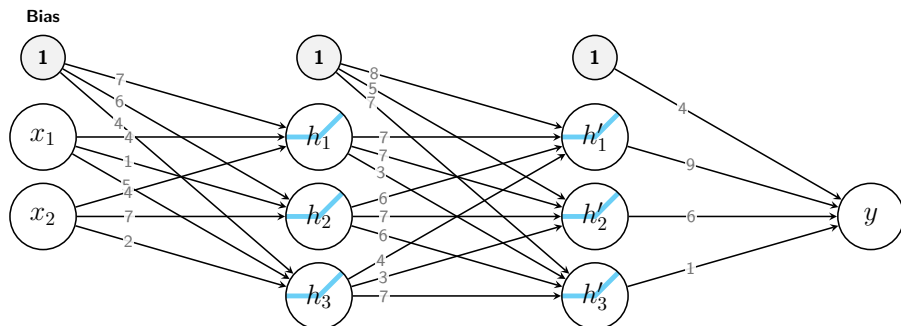


Figure 2: Deep network with 2 inputs

$$h_1 = 5, h_2 = 8.5, h_3 = 0, h'_1 = 94, h'_2 = 99.5, h'_3 = 73, y = 1520$$

Contents

- ① Composing neural networks
- ② From composition to deep networks
- ③ Matrix Notation & General Formulation**
- ④ Shallow vs. Deep Networks

Matrix Notation: Two-Layer Example

Rewriting the equations in Fig 1 using linear algebra.

Layer 1 (Input to Hidden):

$$\begin{bmatrix} h_1 \\ h_2 \\ h_3 \end{bmatrix} = a \begin{bmatrix} \theta_{10} \\ \theta_{20} \\ \theta_{30} \end{bmatrix} + \begin{bmatrix} \theta_{11} \\ \theta_{21} \\ \theta_{31} \end{bmatrix} x, \quad (1)$$

Layer 2 (Hidden to Hidden):

$$\begin{bmatrix} h'_1 \\ h'_2 \\ h'_3 \end{bmatrix} = a \begin{bmatrix} \psi_{10} \\ \psi_{20} \\ \psi_{30} \end{bmatrix} + \begin{bmatrix} \psi_{11} & \psi_{12} & \psi_{13} \\ \psi_{21} & \psi_{22} & \psi_{23} \\ \psi_{31} & \psi_{32} & \psi_{33} \end{bmatrix} \begin{bmatrix} h_1 \\ h_2 \\ h_3 \end{bmatrix}, \quad (2)$$

Output Layer:

$$y' = \phi'_0 + [\phi'_1 \quad \phi'_2 \quad \phi'_3] \begin{bmatrix} h'_1 \\ h'_2 \\ h'_3 \end{bmatrix}. \quad (3)$$

Concept Check

Write the matrix equations for Figure 2.

Concept Check

Write the matrix equations for Figure 2.

First Hidden Layer ($\mathbf{h}$):

$$\begin{bmatrix} h_1 \\ h_2 \\ h_3 \end{bmatrix} = a \begin{bmatrix} 7 \\ 6 \\ 4 \end{bmatrix} + \begin{bmatrix} 4 & 4 \\ 1 & 7 \\ 5 & 2 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}, \quad (1)$$

Second Hidden Layer ($\mathbf{h}'$):

$$\begin{bmatrix} h'_1 \\ h'_2 \\ h'_3 \end{bmatrix} = a \begin{bmatrix} 8 \\ 5 \\ 7 \end{bmatrix} + \begin{bmatrix} 7 & 6 & 4 \\ 7 & 7 & 3 \\ 3 & 6 & 7 \end{bmatrix} \begin{bmatrix} h_1 \\ h_2 \\ h_3 \end{bmatrix}, \quad (2)$$

Output Layer (y):

$$y = [4] + [9 \quad 6 \quad 1] \begin{bmatrix} h'_1 \\ h'_2 \\ h'_3 \end{bmatrix}, \quad (3)$$

Matrix Notation

For a network with D_i inputs, D_o outputs, and K hidden layers.
Let $\mathbf{h}_k$ be the vector of hidden units at layer k .

The General Formulation:

$$\mathbf{h}_1 = a[\boldsymbol{\beta}_0 + \boldsymbol{\Omega}_0 \mathbf{x}]$$

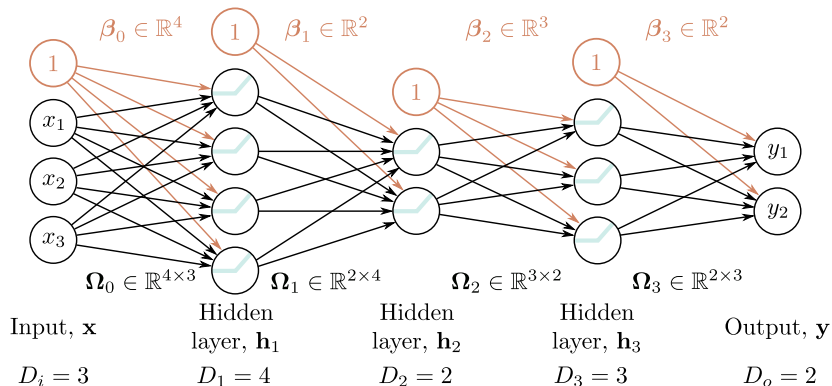
$$\mathbf{h}_2 = a[\boldsymbol{\beta}_1 + \boldsymbol{\Omega}_1 \mathbf{h}_1]$$

$$\vdots$$

$$\mathbf{h}_K = a[\boldsymbol{\beta}_{K-1} + \boldsymbol{\Omega}_{K-1} \mathbf{h}_{K-1}]$$

$$\mathbf{y} = \boldsymbol{\beta}_K + \boldsymbol{\Omega}_K \mathbf{h}_K$$

Visualizing the K-Layer Network



- Ω_k : Weight matrices.
- β_k : Bias vectors.
- Activation function $a[\cdot]$ is applied *element-wise*.

Hyperparameters

The architecture is defined by **Hyperparameters**:

- **Depth (K)**: The number of hidden layers.
- **Width (D_k)**: The number of hidden units in each layer.

The total number of hidden units is a measure of the network's **capacity**.

Note: Hyperparameters are chosen before learning. Parameters (Ω, β) are learned during training.

Contents

- ① Composing neural networks
- ② From composition to deep networks
- ③ Matrix Notation & General Formulation
- ④ Shallow vs. Deep Networks

- **Shallow Networks:** Universal Approximation Theorem states they can model any continuous function arbitrarily closely given enough hidden units.
- **Deep Networks:** Can also approximate any continuous function.

Question: If both can model anything, why use deep networks?

Depth Efficiency:

- **Shallow:** D hidden units $\rightarrow D + 1$ regions using $3D + 1$ parameters.
- **Deep:** Regions grow exponentially with depth.
- K layers of width D can create up to $(D + 1)^K$ regions using $3D + 1 + (K - 1)D(D + 1)$ parameters.

Structured Inputs:

- For high-dimensional inputs like images (10^6 pixels), fully connected layers are impractical.
- Deep learning allows processing local regions and integrating information *hierarchically* (basis for CNNs).

- ① **Architecture:** Multiple hidden layers $K \geq 2$.
- ② **Geometry:** Piecewise linear graph.
- ③ **Matrix notation:** $\mathbf{h} = a[\boldsymbol{\beta} + \boldsymbol{\Omega}\mathbf{h}]$.
- ④ **Efficiency:** Deep networks produce exponentially more linear regions per parameter than shallow networks.
- ⑤ **Practice:** In practice, deep networks are easier to train for complex tasks and generalize better to new data.